

ID2221 Final Project Proposal

WikiPulse: Real-Time Analysis of Wikipedia Edit Streams

Team: Muyang Huang, Yanjun Wang, Hao Tang

1. Problem Description

Wikipedia receives many edits per second across its language editions, and the edit stream tracks current events: breaking news causes bursts of edits on a few articles, controversial topics trigger edit wars of repeated reverts, and bots account for a large share of all changes. We will build WikiPulse, a streaming data platform that ingests the global edit stream and answers four questions in near real time: (1) which articles are trending right now, per language edition; (2) which articles show signs of an edit war; (3) how activity splits between bots and humans, by edition and hour of day; and (4) when the edit rate or revert rate of an edition deviates abnormally from its historical baseline. The deliverable is a working end-to-end pipeline (ingestion, storage, stream and batch processing, serving, visualization), not a single offline analysis.

2. Tools

- **Ingestion and storage:** a small Python producer reads the Wikimedia EventStreams feed and publishes each event to an Apache Kafka topic. Raw events are also archived to HDFS as Delta/Parquet tables so the full history can be reprocessed or replayed.
- **Processing:** Apache Spark (PySpark). Structured Streaming consumes Kafka for the real-time layer; Spark SQL batch jobs over the HDFS archive compute historical baselines (Lambda-style batch and speed layers).
- **Serving:** the Spark jobs write their results to Apache Cassandra, partitioned by (wiki, time window) to suit time-series queries.
- **Visualization:** a Grafana dashboard on Cassandra with live trending pages, potential edit-war alerts, bot/human ratios and an activity heatmap per edition. Streamlit + Plotly is the fallback if the Cassandra plugin proves difficult.
- **Deployment:** one Docker Compose stack on a laptop; no shared cluster needed.

3. Data

We use the public Wikimedia EventStreams recentchange feed (<https://stream.wikimedia.org/v2/stream/recentchange>), a Server-Sent Events stream that needs no API key and emits one JSON event for every change on any Wikimedia project (Wikipedia, Wikidata, Commons, Wiktionary, etc.; about 20-50 events/s in total). Each event carries the wiki id, server_name, title, namespace, user, a bot flag, change type (edit, new, log, categorize), timestamp, edit summary, minor flag and page length before and after. The producer collects the raw stream continuously and archives all of it to HDFS (roughly 2-4 million raw events per day). The analysis set is the subset of Wikipedia article edits, selected by server_name ending in .wikipedia.org; this is a minority of the raw stream (Wikidata and Commons dominate) and its exact share will be measured in the first days of collection. Detailed analyses focus on the largest editions (en, de, fr, zh, sv, ja); per-edition statistics cover all Wikipedia editions.

4. Methodology and Algorithm

- a. Parsing and filtering (streaming):** parse JSON with a fixed schema; keep events with server_name ending in .wikipedia.org, type edit or new, and namespace = 0; derive bytes_changed and is_revert (summary contains "revert", "rv", "undid" or localized equivalents); use a 2-minute event-time watermark.
- b. Trending detection:** 10-minute sliding windows (1-minute slide) grouped by (wiki, title), ranked by the explicit score $\text{trend_score} = \text{edit_count} * \log(1 + \text{distinct_editors})$. This rewards activity from multiple editors so one rapid editor does not dominate.
- c. Potential edit-war alerts (heuristic):** flag an article when a 30-minute window holds at least 3 reverts from at least two distinct non-bot users whose reverts alternate. Reverts also serve legitimate maintenance (e.g. two editors undoing a third party's vandalism), so we treat flags as candidates and check their precision by manual sampling.
- d. Baselines (batch):** daily Spark SQL jobs first aggregate the archive into the same 5-minute per-edition metrics as the streaming side (edit count, revert count, bot share), then compute the mean and standard deviation of each metric per (wiki, hour of day) and write them to Cassandra.
- e. Anomaly detection (streaming):** the streaming job joins each 5-minute per-edition edit count and revert count with its baseline for that hour of day and raises an alert when the z-score exceeds 3.
- f. Evaluation:** replay the same archived data into Kafka at controlled rates (including above natural arrival) so every configuration sees identical input, and compare streaming throughput and end-to-end latency across trigger intervals and Kafka partition counts; precision of edit-war flags by manual sampling; batch-job cost with and without Delta partition pruning.

Deliverables: code (producer, Spark jobs, Cassandra schema, Docker Compose, Grafana dashboard), a 2-page report and run instructions.